Tensors, Data, Probability, and Empirical Risk

Mathematics of Deep Learning — Chapter 3

Yin Yang

Foundation Books Project

Roadmap

Data as Arrays

Tensors, Axes, and Shapes

Datasets, Labels, Batches

Encoding and Normalization

Probability for Classification

Likelihood, Cross-Entropy, Loss

Expected vs. Empirical Risk

Minibatches and Estimates

Practical Data Diagnostics

Summary

Data as Arrays

From Pictures to Numbers

A model never sees "a digit." It receives an array.

- Encoding map $r : \mathcal{X}_{\text{raw}} \to \mathbb{R}^{d_1 \times \cdots \times d_k}$ sends a raw example to a real array.
- A 2×2 grayscale image:

$$A = \begin{bmatrix} 0.00 & 0.50 \\ 1.00 & 0.25 \end{bmatrix} \quad \text{or} \quad x = (0.00, 0.50, 1.00, 0.25).$$

- Same pixels — different shape, different mathematical object.

The translation from image to numbers fixes the input space, the value scale, and the losses that can be optimized.

The Running Example: MNIST-Style Digits

Full MNIST: each grayscale image has 28 rows, 28 columns; label in $\{0, 1, \ldots, 9\}$.

- A modern classifier returns ten numbers — estimated class probabilities.
- This chapter often uses smaller arrays so every step is checkable by hand.
- Failed runs frequently begin with wrong axis order, unexpected pixel scale, mismatched label encoding, or a leaky split.

Before asking "is the model powerful enough," name the object fed to the model and the average computed during training.

Tensors, Axes, and Shapes

Axes Carry Meaning

For $X \in \mathbb{R}^{d_1 \times \cdots \times d_k}$, axis j has length d_j and semantics: examples, channels, height, width, classes, ...

- Shape 64×784 vs. $64 \times 1 \times 28 \times 28$: same pixel count, different interpretations.
- Channel-last (this book): $m \times h \times w \times c$.
- Channel-first (other libs): $m \times c \times h \times w$.

Two arrays with the same total entries are not interchangeable unless the model agrees on which axis is which.

Shape is semantic — the same numbers can mean different objects when axes change.

One Image, Three Layouts

0.00	0.50
1.00	0.25

2×2 image

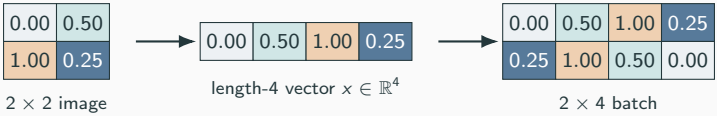
(1) Image carries 2D neighbor structure.

One Image, Three Layouts

(1) Image carries 2D neighbor structure.

(2) Flattening preserves values, changes shape.

One Image, Three Layouts



(1) Image carries 2D neighbor structure.
(2) Flattening preserves values, changes shape.
(3) A batch stacks flattened examples by row.

Flattening throws away spatial layout; convolutional layers later need it back.

A Pocket Shape Table for Images

Object	Shape	Interpretation
One pixel	scalar	grayscale value
One 28×28 image	28×28	rows $\times$ columns
One flattened image	784	feature vector
Batch of flat images	$m \times 784$	one row per example
Channel-last batch	$m \times 28 \times 28 \times 1$	examples, h, w, channels
Channel-first batch	$m \times 1 \times 28 \times 28$	examples, channels, h, w

State the intended shape at each step — and check that the tensor actually has it.

Datasets, Labels, Batches

Examples, Labels, Splits

A supervised image dataset:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}, \qquad x_i \text{ array}, \ y_i \text{ label}.$$

Disjoint index sets $I_{\text{train}}, I_{\text{val}}, I_{\text{test}} \subseteq \{1, \dots, n\}$ define split averages:

$$\widehat{R}_A(\theta) = \frac{1}{|A|} \sum_{i \in A} \ell(f_\theta(x_i), y_i).$$

- Train average: may affect parameter updates.
- Validation average: may affect model selection.
- Test average: read only after all choices are fixed.

Every reported average must say which examples it averages over.

Batches and Batch Axis

A batch groups examples processed together; batch size is the length of the batch axis.

For 64 flattened MNIST images:

$$X_B \in \mathbb{R}^{64\times 784}, \qquad y_B \in \{0,\dots,9\}^{64}, \qquad Y_B \in \{0,1\}^{64\times 10} \text{ (one-hot)}.$$

Row r of X_B and row r of Y_B describe the *same* example.

$$\widehat{R}_B(\theta) = \frac{1}{|B|}\sum_{i\in B} \ell(f_\theta(x_i), y_i).$$

The first axis usually counts examples. A misaligned batch axis silently mismatches inputs to labels.

Encoding and Normalization

Encoding Is Part of the Function

Input encoder $a : \mathcal{X}_{\text{raw}} \to \mathcal{X}$ and label encoder $b : \mathcal{Y}_{\text{raw}} \to \mathcal{Y}$ enter the loss:

$$(x_{\text{raw}}, y_{\text{raw}}) \mapsto \ell\big(f_\theta(a(x_{\text{raw}})),\ b(y_{\text{raw}})\big).$$

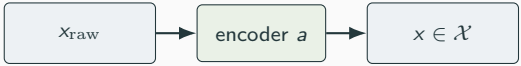
Integer vs. one-hot encoding for K classes:

$$y = 7 \in \{0, \dots, 9\}, \qquad e_7 = (0, 0, 0, 0, 0, 0, 0, 1, 0, 0).$$

Normalization for grayscale: $x_{\text{norm}} = x_{\text{raw}}/255$.

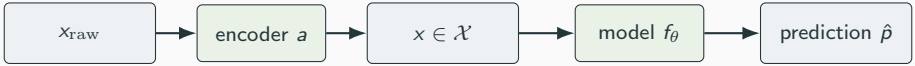
Preprocessing is model design — the codomains of a, b must match the model input and the loss inputs.

Encoding-Composition Check



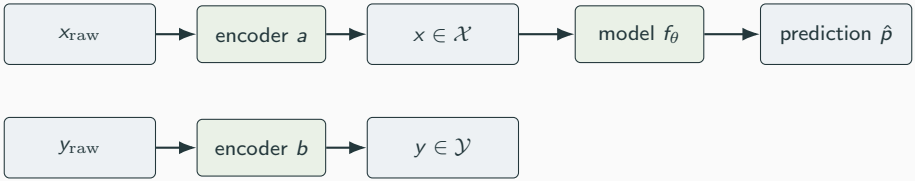
(1) Raw input is encoded into the model's domain.

Encoding-Composition Check



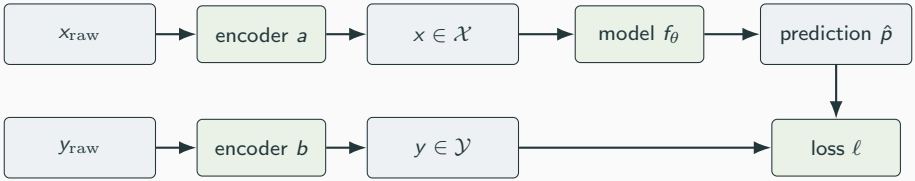
(1) Raw input is encoded into the model's domain.
(2) Model maps domain to its prediction space.

Encoding-Composition Check



(1) Raw input is encoded into the model's domain.
(2) Model maps domain to its prediction space.
(3) Raw label is encoded into the loss's label space.

Encoding-Composition Check



(1) Raw input is encoded into the model's domain.
(2) Model maps domain to its prediction space.
(3) Raw label is encoded into the loss's label space.
(4) Loss reads prediction and encoded label.

A data check is a domain-and-codomain check on this composition.

Split-Safe Standardization

Standardization $a(x) = (x - \mu)/\sigma$.

Training-only statistics, with training values $0, 2$ and validation 10:

$$\mu_{\text{train}} = 1, \quad \sigma_{\text{train}} = 1, \quad a_{\text{train}}(10) = 9.$$

Using all data instead:

$$\mu_{\text{all}} = 4, \quad \sigma_{\text{all}} = \sqrt{56/3}, \quad a_{\text{all}}(10) \approx 1.39.$$

The validation point now looks less unusual — because it helped set μ, σ.

If validation examples shape the preprocessing map, the validation loss no longer estimates held-out risk.

Probability for Classification

Discrete Distributions for Labels

Label as random variable $Y \in \{0, \dots, K-1\}$ with pmf $\mathbb{P}(Y = k) = p_k$.

$$p_k \geq 0, \qquad \sum_{k=0}^{K-1} p_k = 1.$$

Expectation as weighted average:

$$\mathbb{E}[g(Y)] = \sum_{k} g(k)\, p_k.$$

- Bernoulli(π): binary digit-1 vs. digit-0.
- Categorical(p): one class among K digits.
- Conditional: $\mathbb{P}(Y = k \mid X = x)$ given an observed image.

A probability vector has nonnegative entries summing to one — raw scores are not probabilities until they are normalized.

Probabilistic Classifier Output

For a K-class digit classifier:

$$\hat{p} = f_\theta(x) \in [0,1]^K, \qquad \sum_{k=0}^{K-1} \hat{p}_k = 1.$$

Toy four-class output $\hat{p} = (0.05, 0.15, 0.70, 0.10)$.

- Predicted class: $\arg\max_k \hat{p}_k = 2$.
- If $y = 2$: model gave 0.70 to the truth.
- If $y = 1$: model gave only 0.15 to the truth.

Bayes classifier $h^\star(x) \in \arg\max_k \mathbb{P}(Y = k \mid X = x)$ minimizes expected zero-one loss.

The full probability vector carries information that a hard label discards.

Likelihood, Cross-Entropy, Loss

Logits to Probabilities via Softmax

Logits $z \in \mathbb{R}^K$ are unconstrained scores. The softmax map:

$$\text{softmax}(z)_k = \frac{\exp(z_k)}{\sum_{j=0}^{K-1} \exp(z_j)}.$$

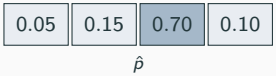
Output lies in the probability simplex

$$\Delta^{K-1} = \left\{ p \in \mathbb{R}^K : p_k \geq 0,\ \sum_k p_k = 1 \right\}.$$

- Softmax outputs are strictly positive $\Rightarrow \log \hat{p}_k$ defined.
- Loss is applied *after* scores become a distribution.

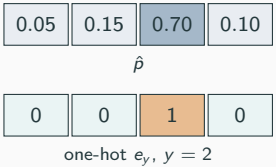
Softmax is the bridge from real-valued scores to a probability vector that a likelihood loss can read.

Cross-Entropy, Step by Step



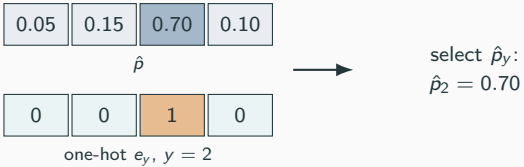
(1) Prediction is a probability vector $\hat{p}$.

Cross-Entropy, Step by Step



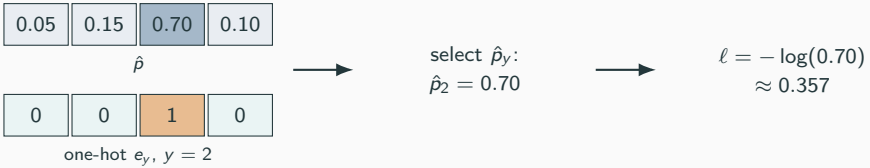
(1) Prediction is a probability vector $\hat{p}$.
(2) Target is the one-hot e_y.

Cross-Entropy, Step by Step



(1) Prediction is a probability vector $\hat{p}$.
(2) Target is the one-hot e_y.
(3) The sum $-\sum_k (e_y)_k \log \hat{p}_k$ keeps the true-class term.

Cross-Entropy, Step by Step



(1) Prediction is a probability vector $\hat{p}$.
(2) Target is the one-hot e_y.
(3) The sum $-\sum_k (e_y)_k \log \hat{p}_k$ keeps the true-class term.
(4) Loss is the negative log of the probability assigned to y.

One-hot cross-entropy reduces to $-\log \hat{p}_y$ — the rest of the vector cancels.

Cross-Entropy Grows When Confidence Is Wrong

Probability assigned to true class	Cross-entropy loss
0.75	0.288
0.25	1.386
0.05	2.996
0.02	3.912

Two models with the same accuracy can still differ in loss — one may assign 0.95 to the truth while the other assigns 0.55.

- Accuracy: was the largest entry correct?
- Cross-entropy: how much probability did you place on the truth?

Cross-entropy rewards confident correctness and punishes confident mistakes.

KL Divergence and Proper Scoring

For target p and model q on K classes:

$$H(p, q) = -\sum_k p_k \log q_k, \quad D_{\mathrm{KL}}(p\|q) = \sum_k p_k \log \frac{p_k}{q_k}.$$

Cross-entropy decomposes:

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p\|q).$$

- $D_{\mathrm{KL}}(p\|q) \geq 0$ with equality iff $q = p$.
- For fixed p, minimizing $H(p, q)$ in q minimizes D_{KL}.
- For $p = e_y$: $H(p) = 0$, $H(p, q) = -\log q_y$.
- D_{KL} is asymmetric — not a distance.

Cross-entropy is a proper scoring rule: the unique minimizer is the truth itself.

Expected vs. Empirical Risk

Two Averages, One Goal

Expected risk (target)
For population distribution P on (X, Y),

$$R(\theta) = \mathbb{E}_{(X,Y)\sim P}[\ell(f_\theta(X), Y)].$$

Empirical risk (computable)
For $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$,

$$\widehat{R}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i).$$

Expected risk is the quantity we care about. Empirical risk is what code can actually evaluate.

Law of Large Numbers Justifies the Proxy

Iid examples $(X_i, Y_i) \sim P$ with $Z_i = \ell(f_\theta(X_i), Y_i)$:

$$\widehat{R}(\theta) = \frac{1}{n}\sum_{i=1}^{n} Z_i \xrightarrow{P} R(\theta).$$

Hoeffding for bounded losses in $[0, 1]$:

$$\mathbb{P}\big(|\widehat{R}(\theta) - R(\theta)| > \epsilon\big) \le 2\exp(-2n\epsilon^2).$$

- True for a *fixed* θ.
- After training, $\widehat{\theta}$ depends on $\mathcal{D}$ — guarantees weaken.
- Held-out validation/test averages restore independence.

Larger n makes the empirical proxy more reliable — but only for a model that was not chosen using the same sample.

Finite-Class Generalization Bound

If $0 \leq \ell \leq 1$ and $\mathcal{F}$ is finite, with probability $1 - \delta$, every $f \in \mathcal{F}$:

$$|R(f) - \widehat{R}_{\mathcal{D}}(f)| \leq \sqrt{\frac{\log(2|\mathcal{F}|/\delta)}{2n}}.$$

- More data n tightens the bound (denominator grows).
- Larger candidate class $|\mathcal{F}|$ loosens it (numerator grows).
- Modern nets aren't analyzed by counting; the same tension remains.
- Refinements use Rademacher complexity, margins, norms, stability.

Finite samples plus a large search space is the basic statistical tension behind "generalization."

Minibatches and Estimates

Minibatch Risk

For index subset $B \subseteq I_{\text{train}}$:

$$\widehat{R}_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \ell(f_\theta(x_i), y_i).$$

Tiny example with full losses $0.2, 0.6, 1.0, 1.4$ (mean 0.8):

Minibatch	Average loss
$\{1, 3\}$	$(0.2 + 1.0)/2 = 0.6$
$\{2, 4\}$	$(0.6 + 1.4)/2 = 1.0$
Full	0.8

A minibatch loss is the same formula, evaluated on fewer rows — cheaper, noisier, but the same kind of average.

Unbiasedness and Variance

Uniform sampling of size-m minibatch B:

$$\mathbb{E}_B[\widehat{R}_B(\theta)] = \widehat{R}(\theta).$$

With-replacement variance:

$$\mathrm{Var}\big(\widetilde{R}_m(\theta)\big) = \frac{\sigma_{\mathcal{D}}^2}{m}, \quad \sigma_{\mathcal{D}}^2 = \frac{1}{n}\sum_{i=1}^{n}(\ell_i - \bar{\ell})^2.$$

- Centered correctly $\Rightarrow$ unbiased estimator.
- Variance scales as $1/m \Rightarrow$ larger batches stabilize.
- Variable per-example losses $\Rightarrow$ noisier estimates.

Unbiased describes the center over many batches; variance describes how far one observed batch can wander.

Worked Minibatch on Three-Class Outputs

Four examples, cross-entropy on the true class:

i	y_i	$\hat{p}_i$	$\ell_i = -\log \hat{p}_{i,y_i}$
1	0	$(0.80, 0.15, 0.05)$	0.223
2	2	$(0.20, 0.30, 0.50)$	0.693
3	1	$(0.10, 0.70, 0.20)$	0.357
4	2	$(0.60, 0.25, 0.15)$	1.897

Full $\widehat{R} \approx 0.793$. Batches: $\{1, 3\} \to 0.290$, $\{2, 4\} \to 1.295$.

The same model gives wildly different minibatch averages — shuffle before forming batches so the sequence is not misleading.

Practical Data Diagnostics

Diagnostics Are Domain Checks

Empirical risk as expectation under the empirical measure:

$$\widehat{P}_{\mathcal{D}} = \frac{1}{n}\sum_{i=1}^{n} \delta_{(x_i,y_i)}, \quad \widehat{R}_{\mathcal{D}}(\theta) = \mathbb{E}_{(X,Y)\sim\widehat{P}_{\mathcal{D}}}[\ell(f_\theta(X), Y)].$$

A diagnostic fails $\Leftrightarrow$ the loss is being read outside the declared $\mathcal{X}, \mathcal{Y}$, or the split.

- Wrong shape: $x \notin \mathcal{X}$.
- Label outside class set: $\ell(\hat{p}, y) = -\log \hat{p}_y$ has no value.
- Leaky preprocessing: $\widehat{\theta}$ no longer independent of held-out data.

A data bug is usually a mathematical domain or codomain mismatch in disguise.

Consistency Checklist

Check	Invariant	Failure mode
Input shape	$x_i \in \mathbb{R}^{784}$ or $\mathbb{R}^{28\times 28\times 1}$	Wrong axis applied to coordinates
Value range	$x_{i,j} \in [0,1]$ after scaling	Model evaluated on a different input domain
Label set	$y_i \in \{0,\dots,K-1\}$	$\hat{p}_{i,y_i}$ undefined
Probabilities	$\hat{p}_{i,k} \geq 0$, $\sum_k \hat{p}_{i,k} = 1$	Loss loses likelihood meaning
Splits	$\widehat{\theta} = A(\mathcal{D}_{\text{train}})$ first	Held-out average conditioned on itself

Verify the empirical measure, spaces, encoding maps, and loss form a well-defined composition before analyzing optimization.

Shape Invariants and Last-Batch Surprises

A shape invariant for a data map T: an equality that holds for every legal example or batch.

$$T : \mathbb{R}^{m \times 28 \times 28} \to \mathbb{R}^{m \times 784}.$$

- m may vary (e.g. last minibatch is smaller).
- 784 is invariant — flattening a 28×28 image.
- Shape $28 \times 28 \times m$ has the same scalar count as $m \times 28 \times 28 \times 1$, but lives in a different product space.

Test pattern: print expected shape, assert actual shape, fail loudly.

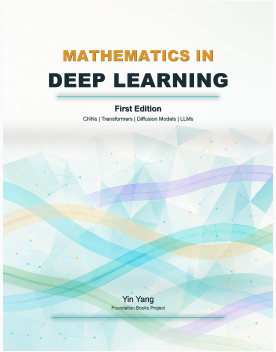
Equal scalar count does *not* mean equal mathematical object — axes carry the structure.

Summary

Chapter in One Slide

- **Arrays:** encoding maps $r : \mathcal{X}_{\text{raw}} \to \mathbb{R}^{d_1 \times \cdots \times d_k}$ turn images into tensors. Shape is semantic.
- **Datasets:** $\mathcal{D} = \{(x_i, y_i)\}$ with disjoint train/val/test indices. Every average must name its examples.
- **Encoding:** integer vs. one-hot labels, $[0, 255] \to [0, 1]$ scaling. Preprocessing is part of the function being evaluated.
- **Probability:** classifier outputs a vector in Δ^{K-1}; softmax bridges logits to the simplex.
- **Loss:** cross-entropy $= -\log \hat{p}_y$ for one-hot targets; proper scoring rule via $H(p, q) = H(p) + D_{\text{KL}}(p \| q)$.
- **Risk:** $R(\theta)$ is the target; $\widehat{R}(\theta)$ the computable sample mean. LLN + Hoeffding link them for fixed θ.
- **Minibatches:** unbiased for $\widehat{R}$, variance $\sigma^2_{\mathcal{D}}/m$.
- **Diagnostics:** domain/codomain checks on the loss composition.

Next: gradients, the chain rule, computational graphs, and the optimization loops that drive empirical risk down.



Mathematics in Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	`foundation-books.github.io/mathematics-in-deep-learning`
Code	`github.com/foundation-books/mathematics-in-deep-learning-code`
Paperback	`amazon.com/dp/B0H1ZZHDT2`
Hardcover	`amazon.com/dp/B0H1ZL11MJ`
Kindle	`amazon.com/dp/B0GX32M8SP`
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.